
boolean



JS - Array

L'Array e i suoi metodi

Ciclo forEach

Un nuovo vecchio ciclo

forEach ci aiuta a ciclare sugli array in maniera più concisa

- **keyword**
  forEach
- **function**
  All'interno delle parentesi tonde inseriamo una funzione che riceverà come argomenti:
  element - l'elemento dell'array sul quale stiamo girando
  index - l'indice di quell'elemento
  array - l'array stesso
- **codice:**
  che verrà eseguito ad ogni giro

```
1  const students = ['Paolo', 'Giulia', 'Marco'];
2
3  students.forEach((element, index, array) => {
4   console.log(element, index);
5  });
6
7
8  // Paolo 0
9  // Giulia 1
10 // Marco 2
```

Ciclo forEach

Un nuovo vecchio ciclo

forEach ci aiuta a ciclare sugli array in maniera più concisa



Questa che tipo di funzione è?

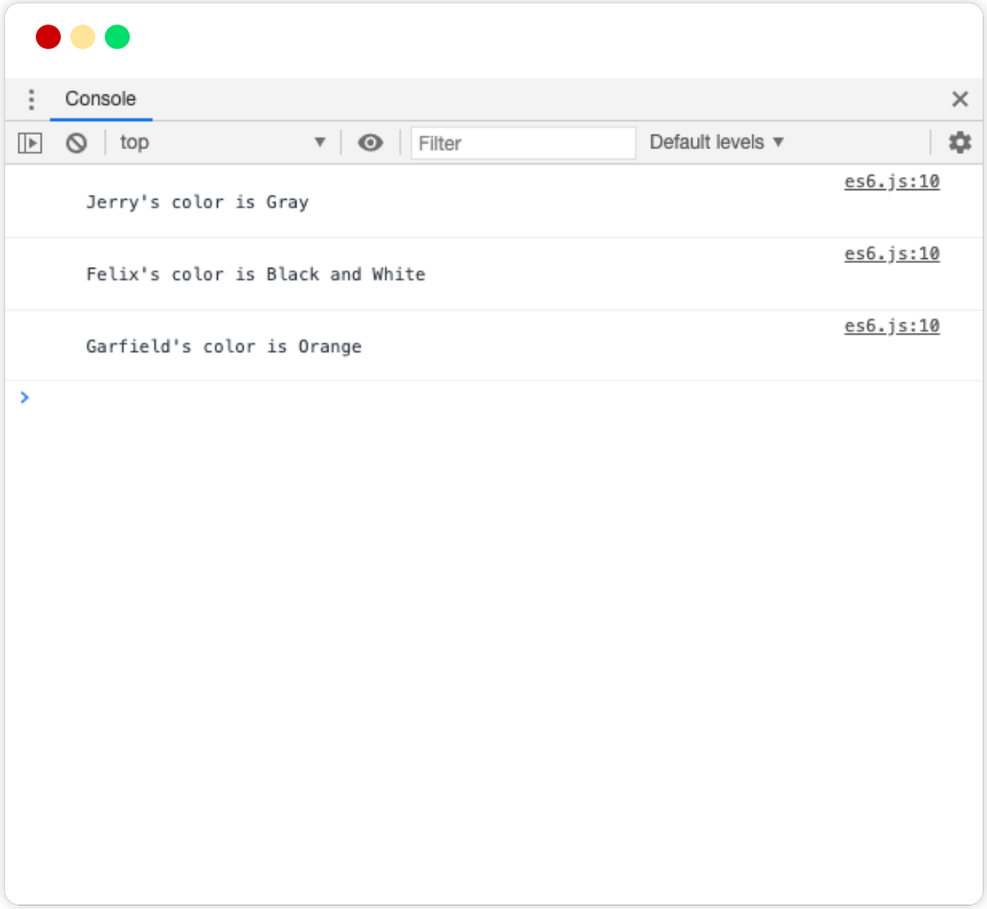
```
1  const students = ['Paolo', 'Giulia', 'Marco'];
2
3  students.forEach((element, index, array) => {
4   console.log(element, index);
5  });
6
7
8  // Paolo 0
9  // Giulia 1
10 // Marco 2
```

Ciclo forEach

Un nuovo vecchio ciclo

Se voglio ciclare su un array di object?

```
1  const cats = [
2   {name: 'Jerry', color: 'Gray'},
3   {name: 'Felix', color: 'Black and White'},
4   {name: 'Garfield',color: 'Orange'},
5  ];
6
7  cats.forEach((element) => {
8   console.log(`
9     ${element.name}'s color is ${element.color}
10  `);
11 });
```

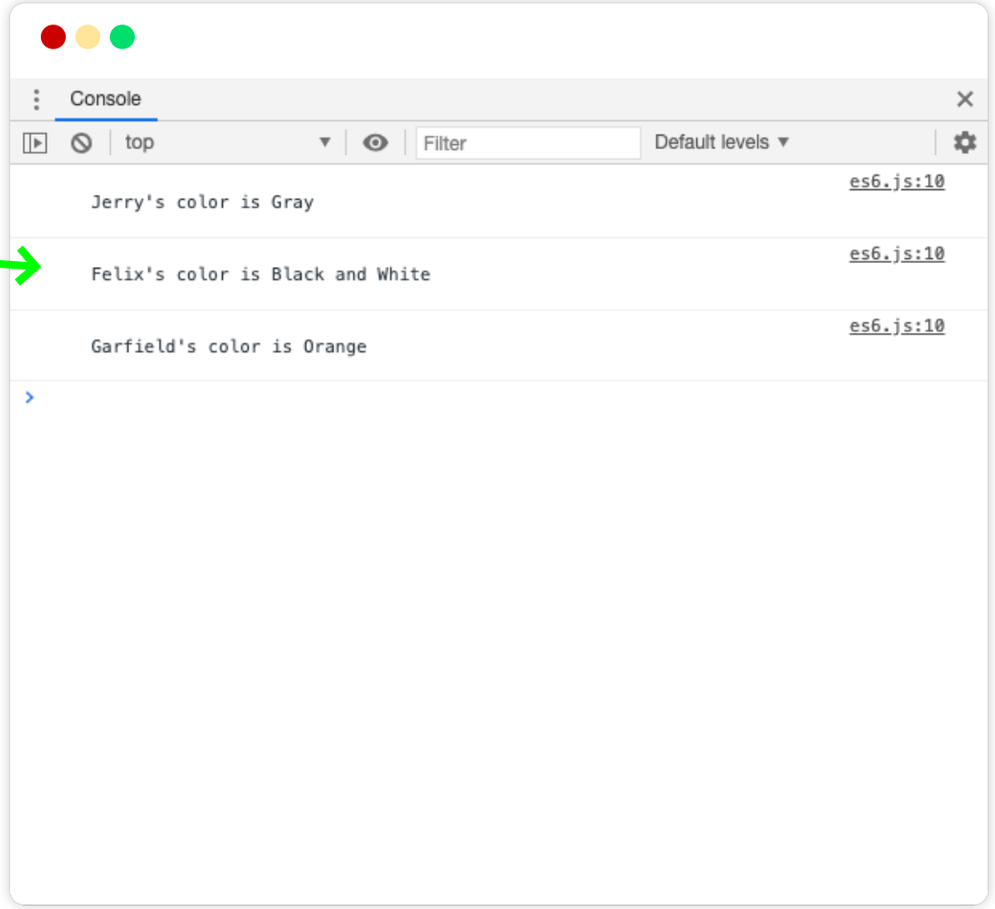


Ciclo forEach

Un nuovo vecchio ciclo

Se voglio ciclare su un array di object?

💡 Come mai abbiamo tutto questo spazio tra una riga e l'altra?



Map, Filter e Find

Alcuni trucchi con gli array



Sappiamo già come iterare sugli elementi degli array, filtrare dati, fare operazioni come sommare tutti gli elementi...

Grazie ad alcuni metodi di JS (*implementati già nella ES5 e ora supportati dalla maggior parte dei browser*) possiamo fare alcune di queste operazioni con più semplicità!

Map

Un ciclo che ci restituisce qualcosa

Map assomiglia molto al forEach...

ma se ne differenzia per il fatto che **ci restituisce un nuovo array** che dobbiamo salvare in una variabile.

```
1 const numeri = [1, 2, 3, 4, 5];
2 const quadrati = numeri.map((numero) => {
3   return numero * numero;
4 });
5 console.log(quadrati); // [1, 4, 9, 16, 25]
```



Quindi cosa succede se metto un return in un forEach?

Filter

E se volessi estrapolare solo alcuni elementi di un array?

Filter ha una sintassi molto simile a quella di Map ma **restituisce solo i valori che verificano una data condizione**.

Come per Map dobbiamo restituire un valore quindi abbiamo bisogno di un return.

```
1 const numeri = [1, 2, 3, 4, 5];
2 const pari = numeri.filter((numero) => {
3   if(numero % 2 === 0) {
4     return true;
5   }
6   return false;
7 });
8 console.log(pari); // [2, 4]
```



Sappiamo farlo con un ciclo for?

Filter

E se volessi estrapolare solo alcuni elementi di un array?

Ritornando a Filter possiamo farlo con meno righe, ma il nostro **return** è un po' particolare...

Dobbiamo inserire la **condizione** che mettiamo nel nostro **If** e **Filter** ci restituirà solo gli elementi che verificano la condizione.



*E se nessuno degli elementi corrisponde alla condizione? Otterremo **un array vuoto***

```
// condizione da verificare
if (numero % 2 === 0) {
 return true;
}
```

```
const numeri = [1, 2, 3, 4, 5];
const pari = numeri.filter((numero) => {
  return numero % 2 === 0;
});
console.log(pari); // [2, 4]
```

Find

E se volessi cercare uno specifico elemento in un array?

Find ha una sitntassi identica a Filter ma **restituisce un solo valore: il primo elemento che supera una data condizione**.

```
1 const numeri = [1, 2, 3, 4, 5];
2 const primoNumeroPari = numeri.find((numero) => {
3   return numero % 2 === 0;
4 });
5 console.log(primoNumeroPari); // 2
```

Dobbiamo inserire la **condizione** nella funzione e se la condizione risulta vera, il ciclo si ferma e l'elemento di turno viene restituito!



*E se nessuno degli elementi corrisponde alla condizione? Otterremo **undefined**!*

```
1 const numeri = [1, 3, 5];
2 const primoNumeroPari = numeri.find((numero) => {
3   return numero % 2 === 0;
4 });
5 console.log(primoNumeroPari); // undefined
```

Find

E se volessi cercare uno specifico elemento in un array?

L'uso più comune per **find** è quello di individuare un elemento in un array di oggetti attraverso il valore di una sua proprietà

Molto spesso, questa proprietà è l'**id**

```
1  const people = [
2      { id: 4, name: 'Paolo', age: 32 },
3      { id: 3, name: 'Clelia', age: 34 },
4  ];
5
6  // trova la person con id 3
7  const myPerson = people.find((person) => {
8      return person.id === 3;
9  });
10
11 //RISULTATO: { id: 3, name: 'Clelia', age: 34 }
```



LIVE CODING